

Today's selected anniversaries:

1906:

The ocean liner RMS Mauretania, the largest and fastest ship in the world at the time, was launched in Tyneside, England. https://en.wikipedia.org/wiki/RMS_Mauretania_%281906%29

1944:

Second World War: Allied forces captured San Marino from the German Army. https://en.wikipedia.org/wiki/Battle_of_San_Marino

1973:

Billie Jean King defeated Bobby Riggs in straight sets at the Astrodome in Houston, Texas, in an internationally televised tennis match dubbed the "Battle of the Sexes". https://en.wikipedia.org/wiki/Battle_of_the_Sexes_%28tennis%29

2011:

The United States military ended its "don't ask, don't tell" policy, consequently allowing gay and lesbian people to serve openly. https://en.wikipedia.org/wiki/Don%27t_ask,_don%27t_tell

Wiktionary's word of the day:

rizz: 1. (transitive, originally US, slang) Often followed by up: to attract or seduce (a love interest). 2. (originally US, slang) A person's ability to attract a love interest, usually female; attractiveness, charm. 3. About Word of the Day 4. Nominate a word 5. Leave feedback <https://en.wiktionary.org/wiki/rizz>

Wikiquote quote of the day:

When the first savage saw his hut destroyed by a bolt of lightning, he fell down upon his face in terror. He had no conception of natural forces, of laws of electricity; he saw this event as the act of an individual intelligence. ... The story of the hero who slays the devouring dragon was not merely a symbol of day and night, of summer and winter; it was a literal explanation of the phenomena, it was the science of early times. --Upton Sinclair https://en.wikiquote.org/wiki/Upton_Sinclair

FRED TALKS

20th September 2026

CONTENTS

Control the ideas, not the code

ANTIREZ.COM · 1245 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

How agents are transforming work | OpenAI

OPENAI · 1582 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

[Daily article] September 20: Misti

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 398 WORDS

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · [SOURCE](#)

Look at the past history of this blog. There are many blog posts about

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even **not** accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn't y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, ... — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter:

[Daily article] September 20: Misti

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 20 SEP 2026 · [SOURCE](#)

Misti is a volcano in the southern Peruvian Andes, rising above Arequipa, Peru's second-largest city. Because it is only 12 km (8 mi) from Arequipa and its million-plus inhabitants, it is considered one of the world's most dangerous volcanoes. Narrow valleys on its western and southern flanks pose particular threats, as mudflows and fast-moving flows of hot volcanic debris could be channelled into the urban area and into important infrastructure, like hydropower plants. The last two significant eruptions were 2,000 years ago and from 1440 to 1470. Even a moderate eruption could deposit volcanic ash and tephra over most of the city. A volcano observatory monitors the mountain. The Inca viewed the volcano as a threat, and during the 1440–1470 eruption offered human sacrifices (capacocha) on Misti and neighbouring mountains to placate the volcano. The mummies on Misti form the largest known Inca human sacrifice. (Full article...).

Read more: <https://en.wikipedia.org/wiki/Misti>

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?
- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?
- If you think your colleagues need to know about this change, because it's a new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.
- Do customers need to know? Who reported the bug? Who's blocked? Let them know.
- Does the world need to know? Announce that it's out.
- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



“How does this apply juniors? You can't expect them to really do all of that?”

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player [Brainstorm](#) app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox

folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally
- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase “ownership” or allude to it, but haven’t explained what “ownership” means to me in a while.

So, ownership.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you...

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?
- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?
- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?
- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven’t confirmed yet?
- Think about how you’d test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?
- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!
- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here’s what I built, under these constraints, with these tradeoffs?”

Daybreak: Tools for securing every organization in the world

SecurityJun 22, 2026



Samsung Electronics brings ChatGPT and Codex to employees

CompanyJun 21, 2026

Ownership

THORSTEN BALL · 08 JUL 2026 · [SOURCE](#)

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses [superpowers-chrome](#) to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called [Slackline](#) - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

How agents are transforming work | OpenAI

OPENAI · 28 JUN 2026 · [SOURCE](#)

June 25, 2026

[Company](#)

A new Economic Research paper measuring Codex's economic potential at the frontier.

[Read the paper](#)

[Listen to article](#)

[AUDIO OMITTED]

[Share](#)

Author

OpenAI

Footnotes

1. Task horizon is estimated using an LLM-as-judge with access to Codex transcripts.

[SVG OMITTED]

[\[find in text\]](#)

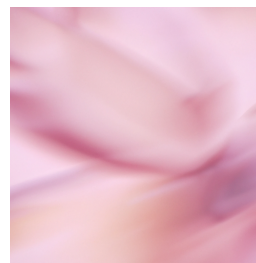
2. These thresholds are model-estimated, so they should be treated as directional rather than exact. This is also based only on individual data and based on the queries of a random sample of 0.1% of users.

[SVG OMITTED]

[\[find in text\]](#)

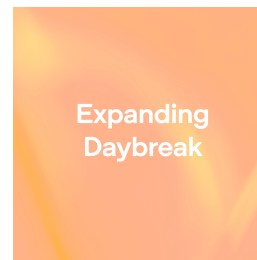
Keep reading

[View all](#)



[OpenAI and Broadcom unveil LLM-optimized inference chip](#)

[Company](#)[Jun 24, 2026](#)



Occupation vs. work done with Codex

Consolidated inferred department	Work category				
	Engineering / Data Coding	Financial Analysis	Knowledge Analysis	Work	Other
Engineering	72%	4%	1%	18%	5%
Data					
Science /	51%	10%	0%	30%	9%
Research					
Finance / Biz Ops	31%	9%	16%	34%	10%
Product /					
Marketing /	25%	3%	7%	51%	15%
Ops					
Other	50%	7%	2%	38%	4%

Share of output tokens within department

Agentic AI changes the unit of knowledge work from single interactions to delegated, long-horizon tasks. Chatbot interactions are often short and self-contained. Agents can operate independently for minutes or hours while orchestrating tool calls, interacting with environments, and iterating towards solutions. As a result, agents are quickly becoming the most powerful AI tool for work.

Over the last year, we witnessed this transformation first-hand at OpenAI. For the first few months after Codex was released to the public, ChatGPT remained the default AI tool for work within OpenAI. Through August 2025, the average OpenAI worker spent less than 10% of their tokens on Codex. Now, every department, including non-technical departments such as Legal and Recruiting, uses Codex as their primary AI tool for work. This pattern reflects what we believe will be the future of work given the expanded capabilities and accessibility of agentic tools.

Some content could not be imported from the original document. [View content ↗](#)

Some content could not be imported from the original document. [View content ↗](#)

Codex adoption grew in tandem with Codex’s capabilities. As Codex leveraged stronger models and new product features, it became capable of taking on an expanding set of productive tasks. Across Individual users, Organizational users, and OpenAI workers, we document four trends over the past year:

- **People use Codex for longer-horizon work.** By May 2026, 80.6% of sampled individual users made at least one Codex request estimated to exceed 30 minutes of human work, 70.2% made one estimated to exceed one hour, and 25.6% made at least one Codex request estimated to exceed eight hours.
- **Codex became the primary AI tool for every department at OpenAI.** Engineering moved first, but Legal, Finance, and Recruiting crossed into Codex being their primary AI tool around April 2026. For the average OpenAI worker, Codex usage now accounts for more than 85% of output tokens. Since Codex users tend to use more tokens than non-users, its share of overall tokens is even higher: Codex accounts for 99.8% of weekly output tokens generated within OpenAI.
- **Non-developer adoption grew especially rapidly, outpacing developer adoption.** Since August 2025, non-developer users rose 137x for individual users, 189x for organizational users, and 12x within OpenAI.

What this means for agents’ economic potential

Increased use of agentic tools by non-engineer employees expands the frontier of what these workers can do. That matters for businesses deciding how to redesign workflows, for employees learning which skills become more valuable, and for policymakers and researchers trying to understand how AI changes the labor market.

Our paper shows how frontier users adopt capable agentic tools at the frontier. Our results demonstrate what unfolds when people have broad, low-friction access to capable agentic tools: as the tools improve, people use them for longer, more complex, and more cross-functional work. As time goes on, this is likely to be what the future of work looks like.

- [2026](#)
- [Economic Research](#)
- [Codex](#)

- **Codex enabled OpenAI workers to do tasks outside their job description.** While technical usage is still most prevalent among engineers, non-technical users regularly use Codex to take on coding or technical execution, including automation, data transformation, tooling, debugging, and structured analysis.

Agents work longer hours on harder tasks

Nearly a quarter of all Codex requests are for tasks that would take a person more than one hour to complete⁽¹⁾. As Codex’s capacity for independent long-context work improved, users shifted from short interactions toward more difficult tasks with longer horizons.

The chart below estimates the share of individual users that crossed four human-time thresholds: tasks that would take a person more than 30 minutes, more than one hour, more than four hours, and more than eight hours⁽²⁾. From December 2025 to May 2026, the share of users who made a request that was estimated to correspond to work that would take a person more than 30 minutes rose to 80.6%. The share making a request that would take a person more than one hour rose to 70.2%. The share requesting work that would take a person more than eight hours grew the fastest from a low base.

The growth in agentic usage can be seen in daily Codex runtime. Among daily active users at OpenAI, the heaviest users ask Codex to run many hours of agent work in a single day. By June 2026, users at the 99th percentile regularly generated more than 60 hours of Codex agent turns per day, distributed across multiple, parallel agents. As Codex became more powerful and parallelizable, users moved from only asking Codex for one answer at a time into increasingly orchestrating multiple agent tasks over the course of a day.

Adoption continues to move from engineers to the rest of OpenAI

Engineers at OpenAI began adopting Codex first, gradually. The average engineer at the company shifted the majority of their usage of OpenAI products to Codex by December 2025. Today, the average engineer generates 99% of their output tokens with Codex rather than ChatGPT. Legal, finance, and recruiting crossed over to majority use of Codex later, around April 2026, but their transitions were much faster. The average lawyer or recruiter at OpenAI now generates more than 85% of their output tokens on Codex.

Over the last six months, Codex usage has deepened and intensified at OpenAI. Among active internal users, change in combined output tokens rose sharply across departments. Research saw the biggest jump: by June 2026, median use was 56 times higher than in November 2025. Customer Support rose 32 times and Engineering rose 27 times, while Legal grew more gradually but still reached 13 times its November level.

These two patterns together illustrate how Codex has transformed how OpenAI uses AI to do productive work. Across the company, users are switching from chatbots to agents as their primary form of AI interaction and are deploying an exponentially growing amount of agentic labor.

Non-developers are the fastest-growing user groups

Across all user groups—OpenAI, organizational, and individual users—Codex use began with developers, the natural target audience for what began as a coding tool. However, as Codex expanded toward more general knowledge work, adoption among non-developers grew even more quickly. As shown in the user growth chart below, weekly non-developer users rose faster than developer users among individual, organizational, and OpenAI populations. By early June 2026, non-developer individual users multiplied 137 times since August 2025. Non-developer organizational users increased 189-fold, and non-developer OpenAI users increased 12-fold, likely because this group already started at a well above average starting point.

The shift does not mean every non-developer is using Codex in the same way as an engineer. Rather, it means that more non-developers are using Codex for some kind of agentic work.

Codex is expanding the horizon of potential productive work

Codex enables non-technical departments to accelerate workflows previously bottlenecked by technical expertise. The heat map below compares inferred occupations within OpenAI to the type of work represented in Codex outputs. Engineering and coding show up as the largest category for data science and research, whereas knowledge work is the largest category for finance and business operations, marketing, operations, and other departments.

That said, agentic tools can expand what an individual worker can do. For instance, over one-fourth of work done with Codex by workers in business functions was engineering or coding. Agents can lower the cost of moving across task boundaries and help workers do adjacent work that used to require more specialized technical support.